

Ingeniería del Software 3

Clase 1 · Cultura DevOps + Git para equipos

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Su instructor — y por qué este curso es como es

Quién soy

- Ing. Ariel Schwindt — Ingeniero de Sistemas (UCC) • Posgrado en Dirección de Sistemas de Información (UB)
- Más de 30 años en el sector: desarrollo, arquitectura de software, gestión de proyectos y gerencia de ingeniería
- Director de Investigación y Desarrollo en TYCON S.A. — integración de aplicaciones, gestión de procesos de negocio y arquitectura de sistemas
- Docente en la UCC: Automatización de Procesos, Integración de Aplicaciones y DevOps
- Microsoft Certified Trainer (MCT) — dicto el curso oficial AZ-400 (DevOps Engineer), base del temario de esta materia

Certificaciones

Microsoft

- DevOps Engineer Expert (AZ-400)
- Azure Developer Associate (AZ-204)
- Azure AI Engineer Associate (AI-102)
- Azure AI Fundamentals (AI-900)
- MCT + Certificación Didáctica (DFCC)

Anthropic (IA aplicada al desarrollo)

- Claude Certified Architect Foundations
- Claude Certified Developer Foundations
- Claude Certified Associate Foundations

Y también: Scrum Master · QA/Testing (UTN)

El curso: el ciclo DevOps completo, con herramientas a tu elección — al final tenés un pipeline real en tu portfolio.

¿Qué vamos a construir este semestre — y para qué?

El problema

"Funciona en mi máquina" no es software entregado. Entre tu código y un usuario real hay un abismo: integrar, probar, empaquetar, desplegar, operar.

En muchos equipos ese abismo lo cruza un héroe, a mano, un viernes a la noche.

Lo que construimos acá

Un SISTEMA que cruza ese abismo solo: cada cambio tuyo se integra, se verifica, se despliega y se observa — automáticamente.

Se llama pipeline, lo armás sobre una app TUYA, y te lo llevás funcionando al final.

- **¿Para qué? Es la habilidad que separa 'sé programar' de 'sé ENTREGAR software'**
- Buscá 'DevOps' o 'CI/CD' en cualquier portal de empleo esta noche: está en casi toda búsqueda de desarrollo
- Te llevás: un pipeline real y público en tu portfolio + la base del temario de la certificación AZ-400

El pipeline del semestre: fases y tecnologías



Debajo de todo: la infraestructura como código (Terraform / Bicep — TP8) · el plan y la trazabilidad (Projects / Boards — TP3) · la IA como copiloto declarado (transversal)

- Cada fase es un TP: la máquina se arma por capas, sobre TU app — y todo corre gratis y sin tarjeta (riel GitHub)
- No hace falta anotar nombres hoy: cada tecnología llega con su clase, su porqué y su guía

La materia: conceptos primero, herramientas después

- El objetivo NO es dominar una herramienta: es entender el enfoque DevOps y poder aplicarlo en cualquier plataforma
- Los detalles finos de cada herramienta los completás con documentación oficial (y con IA)
- El uso de IA está permitido y alentado — con una condición: entender y poder defender lo que entregás
- Se evalúa comprensión: la defensa oral vale más que el repositorio
- **Regla innegociable: si no lo podés explicar, no lo aprobás — aunque funcione**

Evaluación de cada TP: 25% configuración técnica · 25% decisiones.md y evidencias · 50% defensa oral

Cursada 2026: 12 clases

Clases	Qué pasa
C1-C4	Contenido + TP: Git · Docker · Planificación · CI
P1 (clase 5)	Presentación y defensa oral de TPs 1-4
C5-C9	Contenido + TP: Testing · CD · Contenedores+e2e · IaC · DevSecOps
P2 (clase 11)	Presentación y defensa oral de TPs 5-9
R (clase 12)	Recuperatorio: se vuelve a presentar el bloque que quedó desaprobado

- Sin parciales escritos: las notas son dos y salen de las presentaciones (P1 y P2)
- TP Integrador: condición para rendir el final — se construye solo si venís al día
- **Desde el TP2 trabajás sobre UNA app tuya que atraviesa toda la materia**

Cómo se aprueba: regularidad y recuperatorio

Concepto	La condición, sin letra chica
2 presentaciones	P1 (clase 5): TPs 1-4 · P2 (clase 11): TPs 5-9 — cada una lleva UNA nota
Los 9 TPs	No llevan nota propia y no pesan todos igual (ponderación en el README): son lo que defendés en su presentación
Aprobar	Cada presentación con nota ≥ 4 (1-10) — las dos notas van por separado, no se compensan
Recuperatorio	Clase 12 — volvés a presentar el bloque que quedó abajo de 4 (los TPs 1-4 o los 5-9, según cuál sea)
Materia libre	Las DOS presentaciones abajo de 4 — o el recuperatorio tampoco aprobado → recursás
Para el final	TP Integrador validado en vivo en la mesa: al menos un camino vivo y demostrable

El recuperatorio es tu única segunda oportunidad, y alcanza para un solo bloque. Si algo se complica — un servicio caído, una baja, una semana mala — avisá temprano: se acomoda en el momento, no en la clase 12.

Las herramientas las elegís vos (sí, en serio)

Riel	Qué es	Soporte
GitHub (canónico)	Actions + servicios gratuitos SIN tarjeta de crédito. Las demos de clase van por acá	Guía paso a paso completa
Azure	Azure DevOps + Azure (cuenta de estudiante o estándar). El stack de la certificación AZ-400	Tabla de equivalencias + checkpoints
Libre	AWS, GCP, GitLab, lo que quieras — mismo contrato de entregables	Soporte limitado

- **TODO se puede completar gratis y sin tarjeta de crédito**
- Repos públicos obligatorios: habilitan gratis las funciones que vamos a usar (protecciones, CI ilimitado, seguridad)
- Bonus: terminás la materia con un portfolio público real en tu GitHub

¿Qué es DevOps?

"DevOps es la unión de personas, procesos y productos para permitir la entrega continua de valor a nuestros usuarios finales."

— Donovan Brown (Microsoft)

- **Lo que NO dice la definición: herramientas, nubes, pipelines**
- El problema histórico: Desarrollo quiere cambiar el software; Operaciones quiere que no se caiga → objetivos en conflicto
- Resultado de esa separación: entregas gigantes, integraciones traumáticas, deploys heroicos de viernes a la noche
- **La respuesta DevOps: cambios chicos, frecuentes y automatizados**

El ciclo DevOps = el mapa de la materia



Fase del ciclo	Dónde la construís en la materia
Plan	TP3 — boards, historias, trazabilidad
Code	TP1 — Git, branching, Pull Requests (hoy) · TP2 — contenerización de la app
Build + Test	TP4 — CI · TP5 — tests y calidad · TP7 — e2e
Release + Deploy	TP6 — CD y aprobaciones · TP7 — contenedores · TP8 — IaC
Operate + Monitor	TP9 — seguridad, monitoreo, feedback

¿Cómo se mide 'hacer bien DevOps'? – Métricas DORA

- DORA (DevOps Research & Assessment): programa de investigación que estudia miles de equipos desde 2014

Métrica	Qué mide	Equipos de élite
Deployment frequency	Cada cuánto se despliega a producción	Varias veces por día
Lead time for changes	Del commit a producción	Menos de 1 día
Change failure rate	% de deploys que causan fallas	~5%
Failed deployment recovery time (ex MTTR)	Cuánto tarda en recuperarse de un deploy fallido	Menos de 1 hora

Dos de velocidad + dos de estabilidad. La pregunta del millón: ¿se puede tener ambas?

El hallazgo: velocidad y estabilidad van juntas

La intuición

*"Si despliego más seguido,
voy a romper más seguido."*

Velocidad $\rightleftharpoons$ Estabilidad
como trade-off

Los datos

Los equipos que despliegan
**MÁS seguido también
fallan MENOS.**

Las mismas prácticas producen
ambas cosas: cambios chicos,
automatización, buenos tests.

Esta idea es el corazón de la materia: la velocidad sostenible se construye con ingeniería, no con heroísmo.

Control de versiones: la base sobre la que se construye todo

- Un VCS registra quién cambió qué, cuándo y por qué — y permite volver a cualquier estado anterior
- Permite el trabajo en paralelo sin pisarse (el problema real de los equipos)
- **En DevOps es más que historial: es LA FUENTE ÚNICA DE VERDAD**
- Todo lo que automatices este semestre va a vivir como archivos en el repo:
 - pipelines (TP4) • configuración de calidad (TP5) • infraestructura (TP8) • reglas de seguridad (TP9)

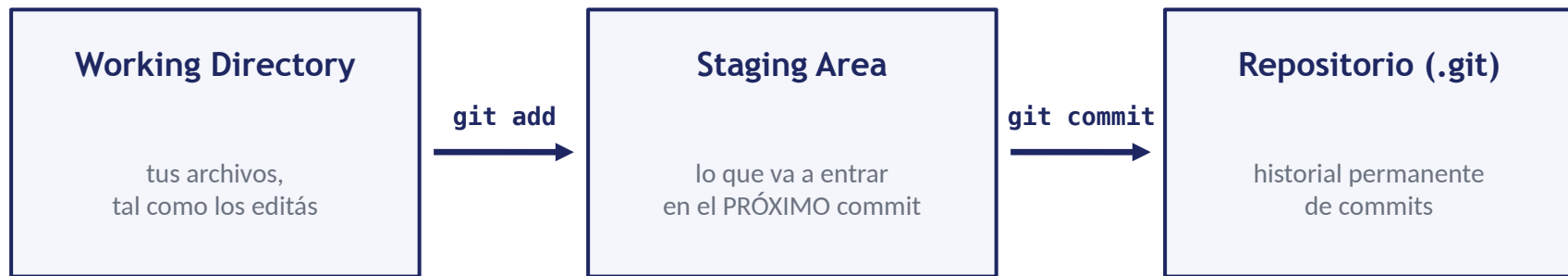
Regla mental de la materia: "si no está en el repo, no existe"

Centralizado vs distribuido: por qué ganó Git

	Centralizado (SVN, TFVC)	Distribuido (Git)
Historial	Solo en el servidor	Completo en cada clon
Sin conexión	Casi nada	Todo: commit, branch, log, diff
Crear una rama	Costoso (copia en el servidor)	Instantáneo (es solo un puntero)
Colaboración	Commit directo / locks	Ramas + merge (Pull Requests)
Hoy	Legacy (TFVC deprecado)	Estándar absoluto

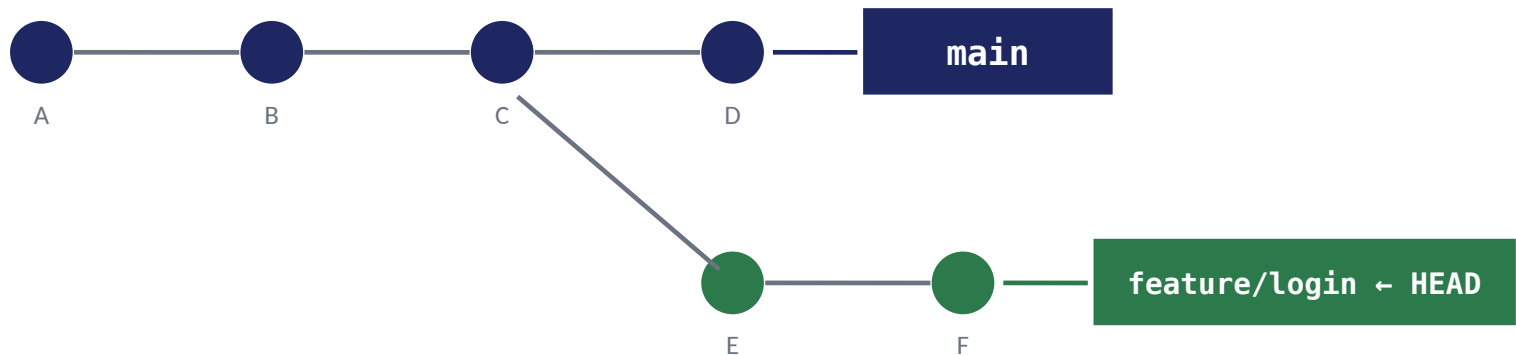
Ramas baratas + historial local = el flujo de trabajo moderno existe gracias a esto

Cómo piensa Git (1): las tres áreas



- Git guarda FOTOS completas del proyecto (snapshots), no diferencias
- ¿Para qué existe la staging area? Para que un commit sea una unidad coherente: de 10 archivos tocados, commiteás los 3 que forman 'un cambio con sentido'
- **Commits chicos y coherentes = historial útil + reviews fáciles**

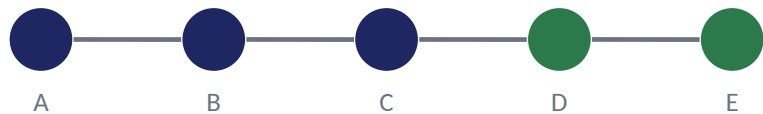
Cómo piensa Git (2): el grafo y las ramas



- El historial es un grafo: cada commit apunta a su(s) padre(s) y tiene un hash único (a1b2c3d...)
- **Una rama NO es una copia: es un puntero móvil a un commit (por eso crearla es gratis e instantáneo)**
- HEAD = en qué rama estás parado ahora
- Local y remoto (origin) son repos completos que pueden divergir — fetch trae novedades, pull = fetch + merge, push publica

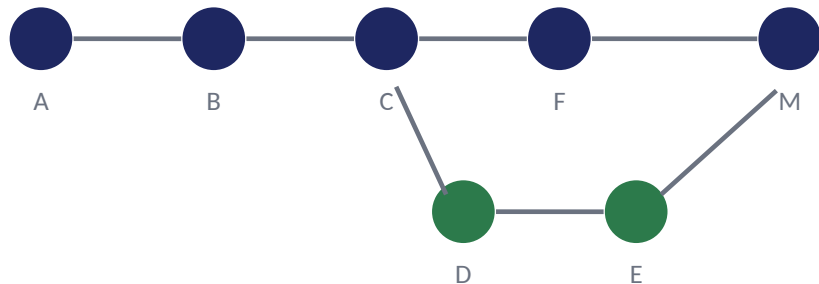
Integrar trabajo (1): fast-forward vs merge commit

Fast-forward — main no avanzó: solo se mueve el puntero



main se mueve de C a E — sin commit nuevo. El historial queda una línea recta

Merge commit (3-way) — AMBAS avanzaron: commit con DOS padres



main también avanzó (F) mientras la feature hacía D y E. M une las dos puntas (F y E) comparándolas con el ancestro común C — por eso 'merge de 3 vías'

El PR de GitHub usa estos mecanismos por debajo — según la opción de merge que elijas

Integrar trabajo (2): las 3 opciones del merge

Tu rama tiene 3 commits arriba de main. La pregunta es qué le llega a main al mergear:

Merge commit



Tus 3 commits entran TAL CUAL — mismo hash — y un cuarto commit deja registrado que hubo dos ramas y se juntaron.

Usalo cuando querés que el historial refleje exactamente cómo se trabajó.

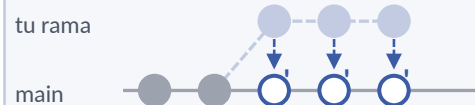
Squash



Los 3 se reemplazan por UNO solo, con el mensaje del Pull Request. El paso a paso interno no queda.

Usalo cuando querés main legible: 1 commit = 1 feature. Es lo más usado hoy.

Rebase



Se vuelven a crear encima de main, como si hubieras trabajado desde ahí: otro hash, y la bifurcación NO queda registrada.

Usalo para historia lineal con detalle — nunca sobre commits ya compartidos.

- **Regla de oro: nunca rebasear commits que ya compartiste con otros**
- No hay opción 'correcta': en el TP1 usás squash (te lo damos) • en el TP4 elegís y justificás

Conflictos: consecuencia normal del trabajo en paralelo

```
<<<<<< HEAD
Proyecto IngSoft3 - versión A
=====
Proyecto IngSoft3 - versión B
>>>>>> feature/titulo-b
```

- Ocurre cuando dos ramas modifican la MISMA línea: Git no puede decidir cuál versión vale
- Resolver = decisión de CONTENIDO, no un comando: elegís qué queda, borras los marcadores, commiteás

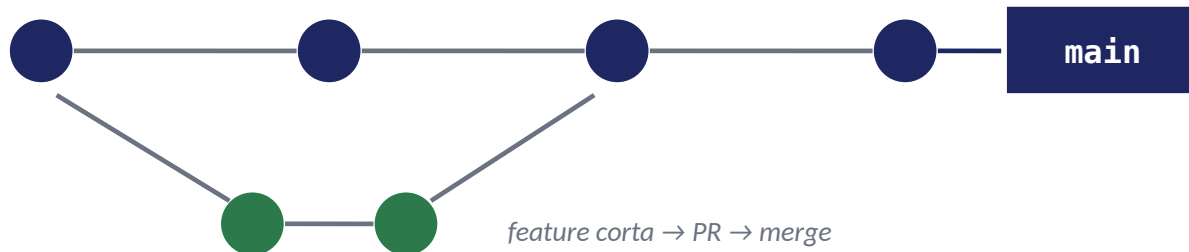
- El conflicto NO es un error — es trabajo en paralelo funcionando
- Lo que SÍ es evitable: el conflicto gigante. Ramas cortas + integración frecuente = conflictos chicos y triviales
- Ramas de semanas = merge hell (y es culpa del proceso, no de Git)

Estrategias de branching: las reglas del juego del equipo

Estrategia	Cómo funciona	Cuándo conviene
Trunk-based	Ramas de horas, integración constante a main; lo no terminado se esconde con feature flags	Equipos maduros con CI fuerte — perfil élite DORA
GitHub Flow ★	Rama por feature → PR → review → merge → deploy	Deploy continuo. Recomendada para la materia
GitFlow	Ramas permanentes main + develop; feature/, release/, hotfix/	Varias versiones en soporte simultáneo

- Es una decisión de EQUIPO: qué ramas existen, cuánto viven y cómo llega el código a main
- Trunk-based vs GitHub Flow: las dos tienen UNA sola rama permanente; cambia cuánto vive tu rama y si el PR es obligatorio
- GitFlow no es 'malo': es la herramienta correcta para OTRO problema (múltiples versiones en producción)

La conexión con DORA: ramas cortas, integración frecuente



- **Cuanto más vive una rama → más tarde se integra → conflictos más grandes → lead time más largo**
- DORA: los equipos de élite integran a main AL MENOS una vez por día
- La estrategia de branching no es estética: mueve directamente 2 de las 4 métricas (lead time y deployment frequency)
- **En el TP1: trabajás con GitHub Flow y protecciones reales • en el TP4 elegís tu estrategia y la justificás**

Pull Requests: el punto de control del flujo

- **Un PR es la propuesta formal de integrar una rama. Empaqueta en un solo lugar:**
 - los commits y el diff completo
 - la conversación de revisión (comentarios, cambios pedidos, aprobaciones)
 - las verificaciones automáticas (desde el TP4: acá corren tus pipelines)
- Es el punto de control de calidad: nada entra a main sin pasar por acá
- **La regla práctica más importante: PRs CHICOS**
 - un PR de 100 líneas recibe review de verdad; uno de 3.000 recibe un 'LGTM 👍' sin leer
 - **si el PR es grande, partilo**

Code review: ¿por qué revisar si 'ya funciona'?

Beneficio	Qué significa
Detección temprana	Un defecto en review cuesta minutos; en producción, horas o días (y usuarios)
Difusión de conocimiento	El reviewer aprende esa parte del código — se elimina el 'solo Juan entiende ese módulo'
Propiedad colectiva	El código deja de ser 'mío' y pasa a ser 'nuestro': el estándar es del equipo

- **Qué mira un buen reviewer:** ¿hace lo que dice? ¿es legible? ¿tiene tests? ¿introduce riesgos? ¿respeto el diseño?
- Qué NO se discute entre humanos: estilo y formato — eso se automatiza (linters)
- **Los TPs son individuales:** el concepto se enseña acá; la práctica con un revisor llega al trabajar en equipo

La IA sí ayuda: revisa cada PR al instante y caza lo mecánico — errores sin manejar, secretos, inyección, tests que faltan.

Pero no reemplaza: no conoce la intención, opina con seguridad aunque se equivoque, y nadie del equipo aprendió nada. El merge lo firmás vos.

Protecciones de rama: la política hecha configuración

- Branch protection convierte el acuerdo ('todo entra por PR revisado') en una regla que la PLATAFORMA hace cumplir
- Patrón de toda la materia — policy as code: los procesos importantes no dependen de la memoria ni de la buena voluntad

Gotcha	GitHub	Azure DevOps
¿Aprobar tu propio PR?	Imposible — NO configurable (API: 422)	Configurable: prohibirlo explícitamente
¿Admin saltea la regla?	Sí, salvo <code>enforce_admins</code>	Sí, con permiso "Bypass policies"
¿Trabajar solo (este TP)?	PR obligatorio + 0 approvals (guía §4.4)	Min reviewers = 1 + 'approve own changes'

Una protección con bypass habilitado es teatro — en el TP: `enforce_admins` + push directo rechazado (evidencia).

Versionado semántico, tags y releases

2.4.1

Parte	Cuándo se incrementa
MAJOR	Cambios incompatibles: 'si actualizás, algo tuyo puede romperse'
MINOR	Funcionalidad nueva compatible
PATCH	Corrección de bugs, sin cambios de comportamiento

- Sufijos: 1.0.0-beta.1 (prerelease) • +build.5 (metadata)
- Tag: nombre inmutable sobre un commit ('acá estaba el código de la v1.0.0') • Release: tag + notas de qué cambió
- **El número de versión es información para el que CONSUME tu software — no decoración**
- La disciplina empieza hoy: TP2 taguea imágenes con semver; en TP6 los deploys nacen de tags

Demo en vivo: el flujo completo

- **Lo que vamos a hacer ahora (guía §4 — seguila después a tu ritmo):**
 - 1. Crear repo público desde github.com/new
 - 2. Proteger main en Settings → Branches (PR obligatorio, 0 approvals, sin bypass) + push directo rechazado
 - 3. Pull Request entero en la web: editar → rama → PR → leer el diff → merge
 - 4. Fabricar un conflicto entre dos ramas y resolverlo con el editor web
 - 5. Tag desde la terminal + release publicada en la web
- ¿Web o consola? La web para ENTENDER qué pasa; los comandos quedan en la guía para cuando quieras automatizar
- **La guía tiene checkpoints  — si un checkpoint no te da, frená y resolvé antes de seguir**

TP1 — Git colaborativo: qué tenés que hacer

- **Escenario:** dejás funcionando el flujo con el que un equipo integra código (los 3 roles los ocupás vos)
- 1 • Repo público + .gitignore acorde al stack
- 2 • main protegida: sin push directo NI PARA VOS — todo entra por PR (0 approvals: el TP es individual)
- 3 • Al menos 2 PRs mergeados:
 - ≥ 1 tiene que haber pasado por un conflicto de merge resuelto
 - Los PRs salen solos: con main protegida, TODO cambio entra por ahí
- 4 • Tag v1.0.0 + release con notas
- La estrategia (GitHub Flow), la convención de ramas y el squash te los damos: elegirlos es tarea del TP4

TP1 — Entregables, defensa y evaluación

Entregable	Contenido
URL del repo público	Historial completo: ramas, PRs con sus conversaciones, conflicto, tag y release (va al formulario de la cátedra)
decisiones.md	Por qué Git no pudo resolver el conflicto solo • problemas encontrados • declaración de uso de IA
evidencias.md	Las 4 capturas 📸 de la guía: push rechazado • aviso de conflicto • marcadores del conflicto • release

- **Defensa oral (en P1, clase 5): preguntas sobre TUS decisiones — en el enunciado hay 5 de EJEMPLO, no es el cuestionario**
- **Evaluación: 25% técnico • 25% decisiones y evidencias • 50% defensa oral**
- Ejemplos: ¿por qué proteger main si 'se tienen confianza'? • ¿qué es una rama REALMENTE? • mostrame tu conflicto

Cierre — para la próxima clase

1

Creá tu repo del TP1 y protegé main

Hoy mismo — los primeros checkpoints de la guía salen en minutos y dejan el flujo armado

2

Completá el TP1 esta semana

La guía tiene checkpoints — si uno no te da, traé la duda a la próxima clase

3

Instalá Docker Desktop / Engine

La clase 2 es 100% hands-on con contenedores

4

Pensá tu app del semestre

Front + back + BD, que corra local, que la entiendas — los criterios finos llegan con la guía del TP2